Software Requirements Specification (SRS)

University Timetable Optimization and Management System

Prepared for: Software Engineering Subject
Document type: Project Proposal + Process Model + SRS + Use Cases + Diagrams

Prepared by: Student Name / Registration No.
Submission date: _____

Version: 1.0

Document Control

Item	Details
Project	University Timetable Optimization and Management System
Document Purpose	To define scope, requirements, actors, process model, use cases, data flows, and visual design diagrams for a university timetable optimization website.
Audience	Course instructor, examiner, student project team, and future developers.
Version	1.0
Status	Draft for academic submission and review.

Table of Contents

1. Project Proposal

1.1 Project Name

University Timetable Optimization and Management System

1.2 What is this system?

The proposed system is a web-based platform that helps a university or department prepare weekly class timetables for teachers, student sections, courses, rooms, and time slots. Instead of manually placing every class into a room and time, the administrator enters academic data, defines scheduling rules, selects policy preferences, and asks the system to generate a timetable draft. The system then checks conflicts, scores timetable quality, supports review and adjustment, and publishes the approved timetable for teachers and students.

1.3 What problem does it solve right now?

The system solves the immediate problem of creating a clash-free and practical university timetable when there are multiple teachers, rooms, classes, student groups, holidays, and policy restrictions. It reduces manual scheduling effort, prevents common clashes, makes timetable quality visible, and supports controlled changes without rebuilding the entire schedule from zero.

1.4 Objectives

- Reduce manual workload in university timetable preparation.
- Prevent teacher, room, section, and timeslot clashes before publication.
- Allow administrators to configure hard constraints and soft preferences without editing source code.
- Support institutional goals such as compact schedules, room proximity, early day ending, energy-aware grouping, and reduced campus movement where selected.
- Support timetable repair after teacher leave, room unavailability, holidays, or approved change requests.
- Provide teachers and students with filtered published timetable views.
- Maintain versions, audit trail, and clear reports for administrative review.

1.5 Explicitly Not in Scope

- Exam timetabling is not included in the first version.
- Individual personalized timetable optimization for every student is not included; the first version schedules by section/group/cohort.
- Transport routing, bus scheduling, hostel allocation, and parking optimization are not included.
- Full multi-campus simulation is not included; building and room proximity may be represented only through simple distance/penalty values.
- Automatic replacement of academic policy decisions is not included; the system supports configuration, but final approval remains with the administrator or department head.
- Deep algorithm comparison and solver benchmarking are not included in this SRS; algorithm design can be handled in a later technical document.

1.6 Main Features

- Academic calendar, holidays, and blocked-period management.
- Course, teacher, section, room, building, and timeslot management.
- Teacher availability and room-suitability recording.
- Hard-constraint and soft-preference configuration.
- Timetable generation from configured data and rules.
- Conflict detection and timetable quality scoring.
- Manual adjustment, locking, and versioned re-optimization.
- Teacher or department change-request workflow.
- Published timetable views for teachers and students.

- Reports for room usage, daily load, late classes, compactness, and resource utilization.
- Export to PDF/Excel/CSV for academic use.

1.7 Users and Needs

User	Main Need	System Support
Admin / Timetable Administrator	Create, review, repair, and publish timetables.	Full access to data, constraints, generation, review, adjustment, versioning, and publication.
Teacher	View teaching timetable and request changes when needed.	Personal timetable view, availability/preference submission, and change-request form.
Student	View official section or group timetable.	Published timetable view filtered by section, semester, course, or day.
Department Head	Review policy choices and approve important changes.	Read-only review, approval support, and quality reports.
Future Facility Manager	Monitor room/resource usage.	Resource reports and building/room utilization summaries if included in a later version.

1.8 System Dependencies

- A database is required to store users, academic data, constraints, timetable versions, and change requests.
- Internet or local network access is required for web-based use.
- A backend API is required to connect the frontend, database, and solver engine.
- A solver/optimization module is required for generating candidate timetables.
- Authentication is required so that only authorized users can change schedules.
- Optional email or notification service may be used for publishing updates and change-request notifications.

2. Process Model Selection

The selected primary lifecycle model is the Incremental Process Model. Prototyping is used only as a supporting validation technique inside early increments, especially for screens, rule configuration, timetable grids, and reports. This avoids the academic weakness of inventing an unclear hybrid model: the process model is Incremental, while prototyping is a technique used within selected increments.

2.1 Selection Criteria

Criterion	Why it matters for this project
Requirement volatility	Timetable rules and preferences often become clearer only after users see draft schedules.
Visual validation	Users need to see forms, timetable grids, conflict reports, and review screens early.
Technical uncertainty	Generation, re-optimization, and minimal-disruption repair introduce risk.
Modular architecture	The system can be developed in slices: data, rules, generation, review, repair, publication.
Coursework documentation	The project needs formal deliverables, diagrams, and traceability.
Semester-scale feasibility	The lifecycle must be disciplined but not too heavy for a student project.

2.2 Candidate Models Considered

Model	Fit	Reason
Waterfall	Low	Best for stable requirements; weak for a timetable system where hidden constraints appear after draft review.
Incremental	High	Builds useful modules in controlled slices and allows later increments to

		absorb feedback from earlier ones.
Spiral	Medium	Strong for high-risk projects but too management-heavy for a semester-scale academic project.
Agile/XP practices	Medium	Useful for short feedback loops inside increments, but too broad as the formal lifecycle label for this report.

2.3 Chosen Model Justification

- The system is modular: calendar/data setup, constraints, generation, review, repair, and publishing can be built separately.
- The first generated timetable may reveal missing rules, so feedback must be absorbed without restarting the entire project.
- The generation MVP should appear early enough to expose algorithmic and data-model risks.
- Re-optimization after change requests is a natural later increment, not an afterthought.
- Documentation remains possible because each increment produces updated requirements, test cases, and design artifacts.

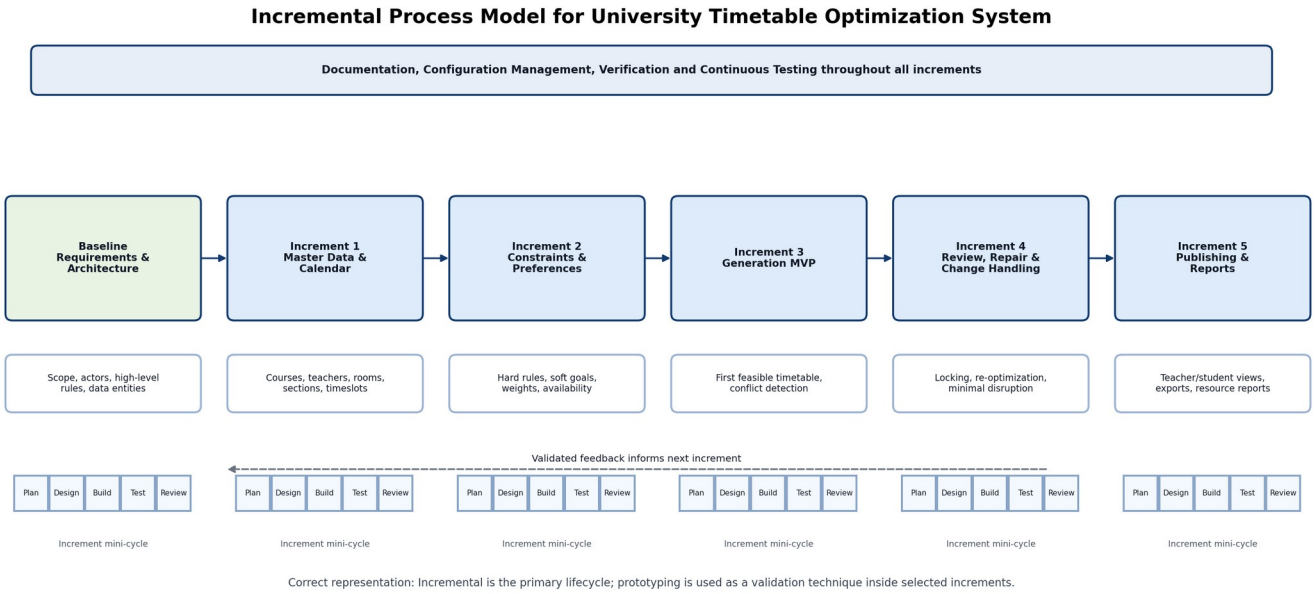


Figure 1. Incremental process model with internal mini-cycles and continuous documentation/testing.

2.4 Diagram Description

Figure 1 shows a baseline requirements and architecture stage followed by five functional increments. Each increment contains a small internal cycle of planning, design/prototyping, building, testing, and review. The top band shows activities that continue throughout development: documentation, configuration management, verification, and continuous testing. This representation is academically safer because it keeps Incremental as the primary lifecycle and shows prototyping as a validation technique rather than as a separate mixed lifecycle.

3. Requirements Specification

3.1 Purpose

The purpose of this SRS is to define the functional and non-functional requirements of the University Timetable Optimization and Management System. The document gives enough detail for software engineering analysis,

design diagrams, use cases, and later implementation planning, while avoiding deep algorithm design at this stage.

3.2 Scope

The first version supports weekly course/class timetabling for one university department or faculty. It manages academic data, constraints, timetable generation, review, repair after changes, and publication. The system handles teachers, courses, sections, rooms, time slots, academic calendar, holidays, and timetable versions.

3.3 Definitions and Abbreviations

Term	Meaning
Admin	The timetable administrator who operates the scheduling system.
Section	A student group/cohort taking a set of courses in a semester.
Timeslot	A defined day and time range in which a class can be scheduled.
Hard Constraint	A rule that must not be violated, such as no teacher clash or no room double-booking.
Soft Preference	A desirable goal that can be violated with penalty, such as early finish or room proximity.
Solver Engine	The optimization component that assigns classes to rooms and timeslots.
Timetable Version	A saved draft or published copy of a timetable.
Re-optimization	The process of repairing an existing timetable while minimizing unnecessary changes.

3.4 Product Perspective

The system is a web application with a frontend interface, backend API, relational database, and solver engine. The frontend is used by administrators, teachers, students, and department heads. The backend enforces validation, permissions, workflow, and integration with the solver. The database stores academic records and timetable versions. The solver engine produces candidate assignments from the problem model sent by the backend.

3.5 User Characteristics

User	Expected Skill Level	Implication for Design
Admin	Moderate computer literacy and timetable knowledge.	Needs clear forms, validation errors, preview screens, and editable rule configuration.
Teacher	Basic web use.	Needs simple view, availability/preference entry, and change-request submission.
Student	Basic web/mobile use.	Needs fast access to a readable timetable without complex settings.
Department Head	Review-oriented user.	Needs summaries, quality scores, approvals, and policy visibility.

3.6 General Constraints

- The system should be developed within a semester-scale student project timeline.
- The first version should use a simple web architecture rather than an enterprise-scale multi-campus platform.
- The system should work on normal university lab computers and common browsers.
- The solver should be replaceable later without rewriting the entire frontend.
- The final report should remain understandable to a software engineering examiner, not only to optimization experts.

4. Functional Requirements

Every requirement in this section uses the required format: “The system shall ...”. The requirements are grouped by high-level use case.

UC-01 Maintain Scheduling Data

- The system shall allow the Admin to add, update, view, and disable courses with course code, course name, credit hours, weekly session count, and department.
- The system shall allow the Admin to maintain teacher records with name, department, contact information, teaching load, and availability status.
- The system shall allow the Admin to maintain rooms with building, floor, capacity, room type, and available facilities such as lab equipment or projector.
- The system shall allow the Admin to maintain sections/student groups with program, semester, student count, and assigned courses.
- The system shall allow the Admin to define academic calendar dates, holidays, working days, and blocked periods.

UC-02 Generate Timetable

- The system shall allow the Admin to start timetable generation for a selected academic term and department.
- The system shall prevent two classes from being assigned to the same room at the same timeslot.
- The system shall prevent a teacher from being assigned to more than one class at the same timeslot.
- The system shall prevent a student section from being assigned to more than one class at the same timeslot.
- The system shall generate a draft timetable using active hard constraints and selected soft preferences.
- The system shall display generation progress, completion status, and failure messages if no feasible timetable can be found.

UC-03 Review and Adjust Timetable

- The system shall show the generated timetable in day-wise and section-wise views.
- The system shall display conflict counts, hard-constraint violations, soft-preference penalties, and room utilization summaries.
- The system shall allow the Admin to manually adjust selected timetable entries when permitted.
- The system shall allow the Admin to lock selected classes so that later re-optimization does not change them.
- The system shall validate every manual change before saving it.

UC-04 Publish and View Timetable

- The system shall allow the Admin to publish only an approved timetable version.
- The system shall allow Teachers to view their assigned teaching timetable.
- The system shall allow Students to view the published timetable for their section, semester, or group.
- The system shall allow users to filter the published timetable by day, course, teacher, room, section, or department.
- The system shall support exporting the published timetable in printable or downloadable format.

UC-05 Manage Change Requests and Re-optimize

- The system shall allow Teachers to submit timetable change requests with reason, affected class, preferred alternative time, and urgency.
- The system shall allow the Department Head or Admin to approve, reject, or return change requests for clarification.
- The system shall allow the Admin to re-optimize the timetable after an approved change request while preserving locked and accepted entries where possible.
- The system shall store each re-optimized timetable as a new version.
- The system shall show the difference between the previous timetable version and the revised version before publication.

4.6 Level 3 SRS Function Table

Level 3 SRS - Maintain Scheduling Data

Field	Detail
Function	Maintain Scheduling Data
Description	Admin records academic and resource data required for scheduling.
Input	Courses, teachers, rooms, sections, timeslots, calendar.
Source	Admin / Department Head
Output	Stored scheduling data.
Destination	Database
Requires	Authorized login and valid form data.
Pre-condition	Admin is logged in.
Post-condition	Data becomes available for generation.
Side Effects	Invalid data may block generation until corrected.

Level 3 SRS - Generate Timetable

Field	Detail
Function	Generate Timetable
Description	System creates a timetable draft from data, constraints, and preferences.
Input	Term, department, constraints, weights, active data.
Source	Admin
Output	Draft timetable, score, conflict report.
Destination	Admin / Database
Requires	Scheduling data and constraints.
Pre-condition	Required data exists and validation passes.
Post-condition	Draft version is stored for review.
Side Effects	High computation load during generation.

Level 3 SRS - Review and Adjust Timetable

Field	Detail
Function	Review and Adjust Timetable
Description	Admin reviews draft quality and edits or locks selected entries.
Input	Draft timetable, conflicts, manual changes.
Source	Admin / Department Head
Output	Adjusted timetable version.
Destination	Database
Requires	Existing draft timetable.
Pre-condition	Draft has been generated.
Post-condition	Revised version is saved or marked for publication.
Side Effects	Changes may require re-validation.

Level 3 SRS - Publish Timetable

Field	Detail
Function	Publish Timetable
Description	Approved timetable is released to teachers and students.
Input	Approved version ID.
Source	Admin
Output	Published timetable views and export files.
Destination	Teachers / Students
Requires	Reviewed and approved timetable.
Pre-condition	Timetable status is approved.
Post-condition	Users can view official timetable.
Side Effects	Previous version becomes archived.

Level 3 SRS - Manage Change Request

Field	Detail
Function	Manage Change Request
Description	Teacher or department submits and tracks schedule change request.
Input	Affected class, reason, preferred time, urgency.
Source	Teacher / Department Head
Output	Approved/rejected request and possible revised timetable.
Destination	Admin / Database
Requires	Published or draft timetable.
Pre-condition	Requester is authenticated.
Post-condition	Request is logged and acted upon.
Side Effects	May trigger re-optimization and notifications.

5. Non-Functional Requirements

5.1 Performance

- The system shall open common pages such as login, dashboard, and timetable view within 3 seconds under normal university network conditions.
- The system shall save ordinary data-entry forms within 2 seconds under normal load.
- The system shall generate a small departmental timetable draft within 5 minutes for typical semester data.
- The system shall show a progress indicator if timetable generation takes more than 10 seconds.
- The system shall allow the Admin to cancel or retry generation if the solver cannot produce a result within the configured limit.

5.2 Security

- The system shall require authenticated login for all administrative, teacher, and department-head functions.
- The system shall restrict timetable creation, generation, approval, and publishing to authorized Admin users only.
- The system shall allow Teachers to view only their own timetable and submit only permitted change requests.
- The system shall allow Students to view only published timetable information, not draft versions or administrative controls.
- The system shall record audit logs for timetable generation, manual edits, approvals, publication, and change-request decisions.

5.3 Usability

- The system shall provide clear labels, validation messages, and simple navigation for non-technical users.
- The system shall display timetable information in familiar grid form by day and time.
- The system shall show conflicts and rule violations in plain language, not only as numeric scores.
- The system shall allow constraint weights and preferences to be configured through forms rather than through source-code changes.
- The system shall provide search and filtering for teachers, courses, rooms, sections, and days.

5.4 Availability

- The system shall be available during university working hours for timetable administration and timetable viewing.
- The system shall keep the last published timetable available even if a new draft generation fails.
- The system shall protect published timetable versions from accidental deletion.
- The system shall support database backup so that timetable data can be restored if needed.

6. Constraints and Assumptions

6.1 Project Constraints

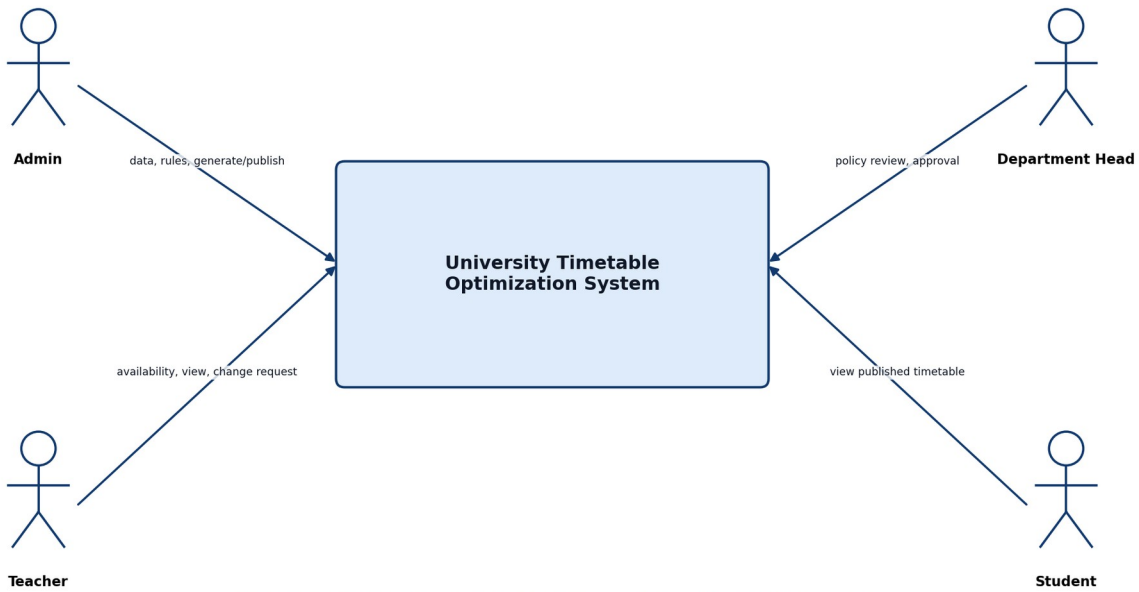
Constraint Type	Constraint
Hardware	The system should run on ordinary university lab PCs or a standard cloud/server machine.
Platform	The user interface should work in common modern browsers.
Budget	The first version should use free or student-accessible tools where possible.
Deadline	The scope must remain achievable within a semester project timeline.
Database	A relational database should be used for structured academic records and timetable versions.
Algorithm	The SRS defines solver requirements but does not finalize the optimization algorithm.
Data	Accurate teacher, room, course, section, and calendar data must be supplied before reliable generation is possible.

6.2 Assumptions

- The first implementation targets one department or faculty rather than the entire university.
- Teachers and sections follow a weekly recurring timetable pattern.
- The Admin has authority to publish the final timetable.
- The Department Head can review or approve changes according to local policy.
- Holidays and blocked periods are known before generation or can be added later through re-optimization.
- The solver may return a feasible but not perfect timetable; the Admin can review, adjust, and improve it.

7. Data Flow and Architecture

Level 0 Context Diagram



Context diagrams show the system as one black box; internal processes are not shown here.

Figure 2. Level 0 context diagram.

Figure 2 shows the system as one black-box boundary. This is the correct representation for a context diagram: external actors are outside the system and interact with the system through high-level data or commands.

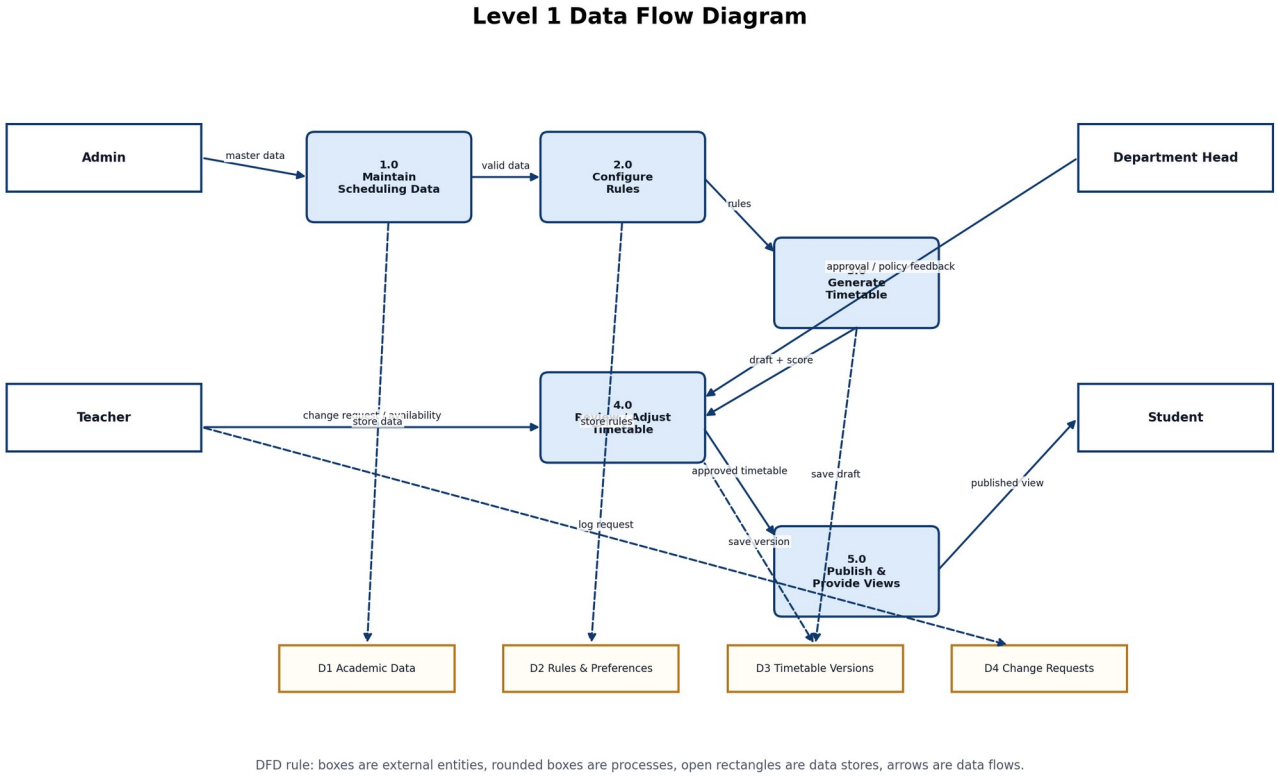


Figure 3. Level 1 data flow diagram.

Figure 3 decomposes the black-box system into the main data processes: maintaining scheduling data, configuring rules, generating timetables, reviewing/adjusting drafts, and publishing views. It also identifies the main data stores: academic data, rules/preferences, timetable versions, and change requests.

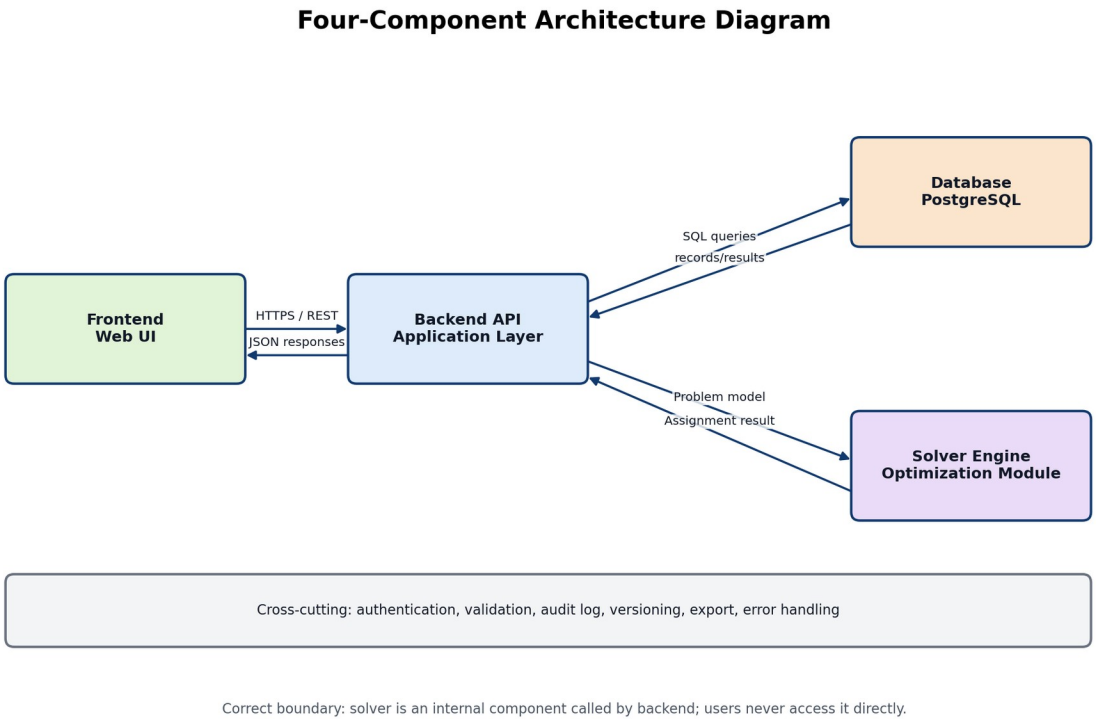


Figure 4. Architecture diagram with four core components.

Figure 4 shows the implementation architecture. The frontend communicates with the backend API through HTTPS/REST. The backend reads and writes persistent data through SQL queries. It sends a problem model to the solver engine and receives assignment results. The solver is an internal service/module, not a direct user-facing component.

8. Use Cases

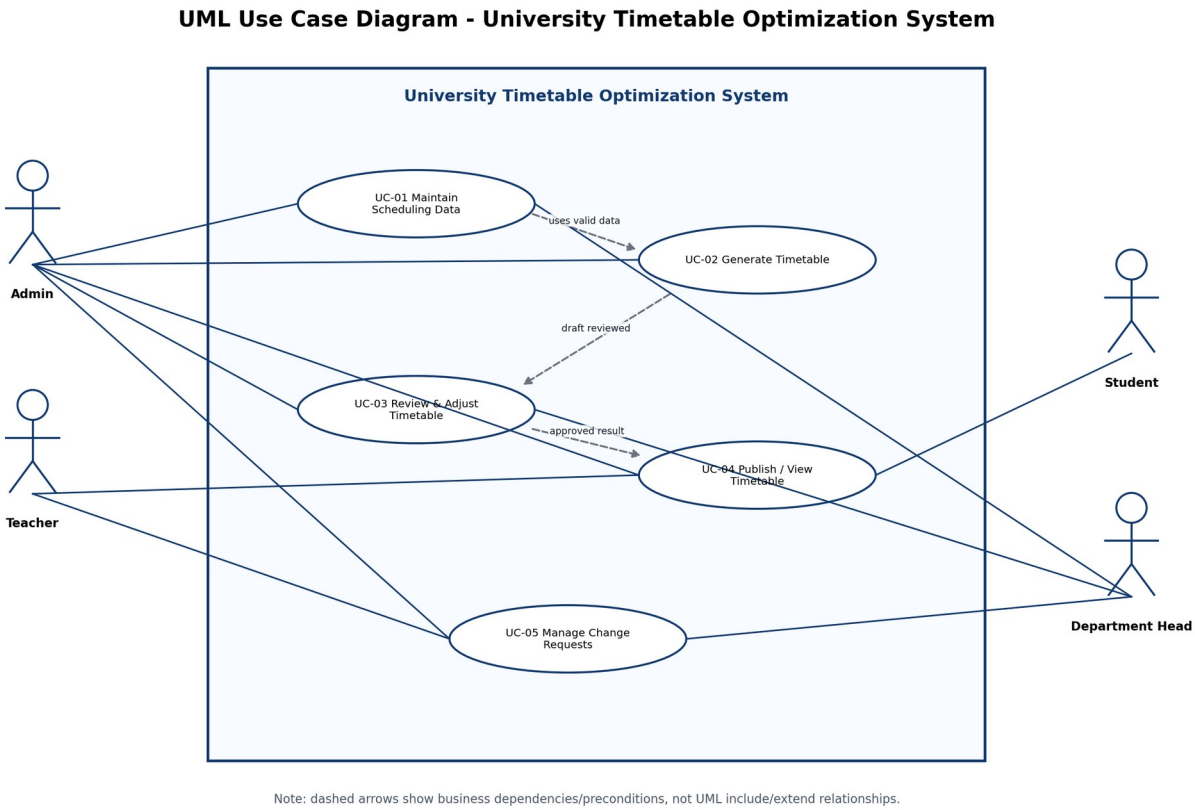


Figure 5. UML use case diagram for UC-01 through UC-05.

8.1 Diagram Description

Figure 5 contains one clear system boundary labelled University Timetable Optimization System. Actors remain outside the boundary. Use cases remain inside the boundary. Direct lines show actor participation. Dashed arrows are labelled as business dependencies/preconditions, not as UML include/extend relationships, because generation depends on configured data and publishing depends on approval, but these are not reusable sub-use cases executed inside another use case.

8.2 Use Case Catalogue

ID	Use Case	Actors	Type	Essential Goal	Real Meaning
UC-01	Maintain Scheduling Data	Admin, Department Head	Primary	Keep academic and resource data ready for scheduling.	The Admin enters courses, teachers, rooms, sections, timeslots, calendar, and holidays; the Department Head may validate policy-sensitive data.
UC-02	Generate Timetable	Admin	Primary	Produce a feasible timetable draft.	The Admin starts generation and the system assigns classes to timeslots and rooms while respecting

					constraints.
UC-03	Review and Adjust Timetable	Admin, Department Head	Primary	Assess and improve generated timetable quality.	The Admin and Department Head review conflicts, scores, room usage, and policy outcomes before acceptance.
UC-04	Publish / View Timetable	Admin, Teacher, Student	Primary	Release and access the official timetable.	The Admin publishes the approved version; teachers and students view filtered official timetables.
UC-05	Manage Change Requests	Teacher, Admin, Department Head	Primary	Submit, approve, and handle timetable changes.	A teacher requests a change; the Department Head/Admin reviews it; the Admin may re-optimize with minimum disruption.

9. Expanded Use Cases

UC-01 Maintain Scheduling Data

Field	Detail
Actors	Admin, Department Head
Purpose	To maintain the academic and resource information required before timetable generation.
Pre-condition	Admin is logged in and has data-maintenance permission.
Post-condition	Valid academic, room, section, teacher, calendar, and timeslot data are stored.
Main Flow	Admin opens the scheduling data module; system displays forms; Admin adds or updates records; system validates required fields and uniqueness; system saves records; Department Head may review policy-sensitive data.
Alternative Flow	If data is incomplete or invalid, the system displays field-level errors and prevents saving until corrected.

UC-02 Generate Timetable

Field	Detail
Actors	Admin
Purpose	To create a feasible timetable draft from configured data and rules.
Pre-condition	Scheduling data, constraints, calendar, and timeslots exist and pass validation.
Post-condition	A draft timetable version is generated or a clear failure report is produced.
Main Flow	Admin selects term and department; system loads active data and constraints; Admin confirms generation settings; system validates inputs; system builds the problem model; solver creates candidate assignment; system stores draft version; system shows timetable, score, and conflicts.
Alternative Flow	If data is missing or infeasible, the system stops generation and shows reasons such as missing room, teacher clash, or

	impossible availability constraint.

UC-03 Review and Adjust Timetable

Field	Detail
Actors	Admin, Department Head
Purpose	To evaluate timetable quality and make controlled improvements.
Pre-condition	A draft timetable exists.
Post-condition	Draft is accepted, revised, locked partially, or sent back for re-generation.
Main Flow	Admin opens generated draft; system displays timetable grid and quality report; Admin reviews conflicts and soft penalties; Department Head reviews policy outcomes; Admin edits or locks selected entries; system validates changes and stores a revised version.
Alternative Flow	If a manual adjustment creates a clash, the system rejects it or marks it as unresolved until corrected.

UC-04 Publish / View Timetable

Field	Detail
Actors	Admin, Teacher, Student
Purpose	To publish the approved timetable and make it available to users.
Pre-condition	A timetable version has been reviewed and approved.
Post-condition	Teachers and students can view the official published timetable.
Main Flow	Admin selects approved version; system asks for confirmation; Admin publishes; system marks version as published; teachers and students log in or open view page; system shows filtered timetable.
Alternative Flow	If no approved version exists, the system prevents publication and shows a warning.

UC-05 Manage Change Requests

Field	Detail
Actors	Teacher, Admin, Department Head
Purpose	To handle timetable changes after a draft or published timetable exists.
Pre-condition	Requester is authenticated and a timetable entry exists.
Post-condition	Request is approved, rejected, returned, or used to create a revised timetable version.
Main Flow	Teacher submits request with reason and affected class; system stores request; Department Head/Admin reviews request; Admin chooses re-optimization or manual adjustment; system tries to preserve locked entries and minimize unnecessary changes; revised version is reviewed before publication.
Alternative Flow	If the requested change is impossible, the system shows impact reasons and the request can be rejected or revised.

10. System Sequence Diagram

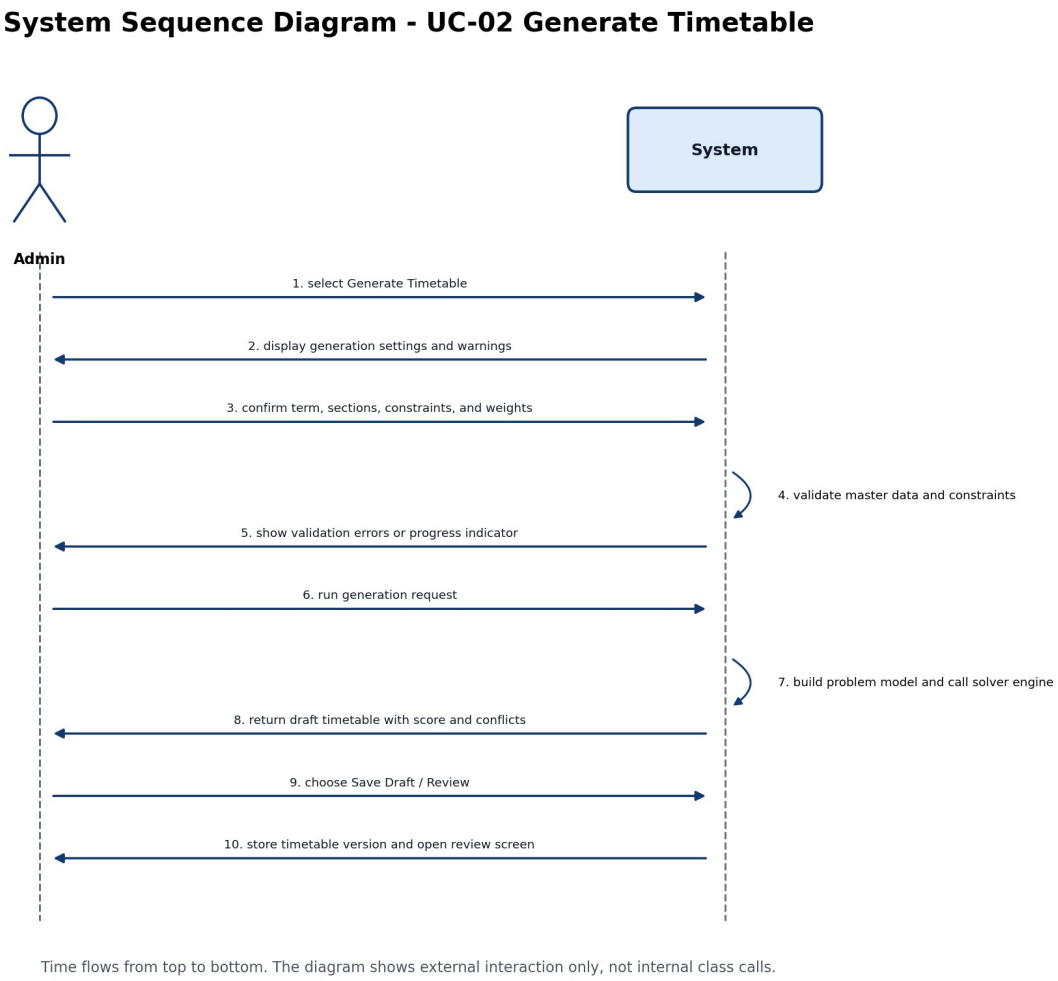


Figure 6. System sequence diagram for UC-02 Generate Timetable.

Figure 6 shows only the interaction between the external actor and the system for UC-02. It deliberately does not show internal classes, database tables, or solver internals. Time flows from top to bottom. This is the correct abstraction for a system sequence diagram.

11. Workflow and Domain Model

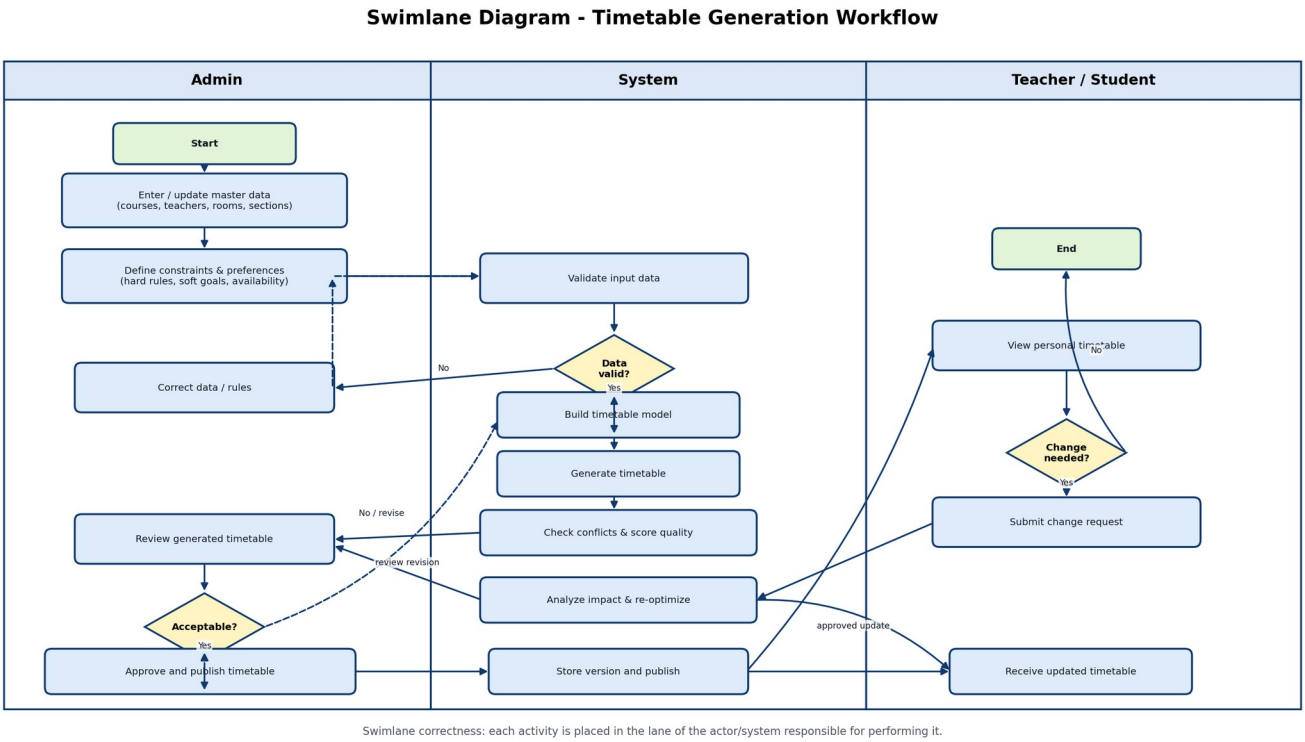


Figure 7. Swimlane workflow for timetable generation and revision.

Figure 7 separates responsibilities across Admin, System, and Teacher/Student lanes. The Admin enters data, defines rules, reviews drafts, and approves revisions. The System validates, generates, scores, stores, publishes, and re-optimizes. Teachers and students view timetables and submit change requests when needed.

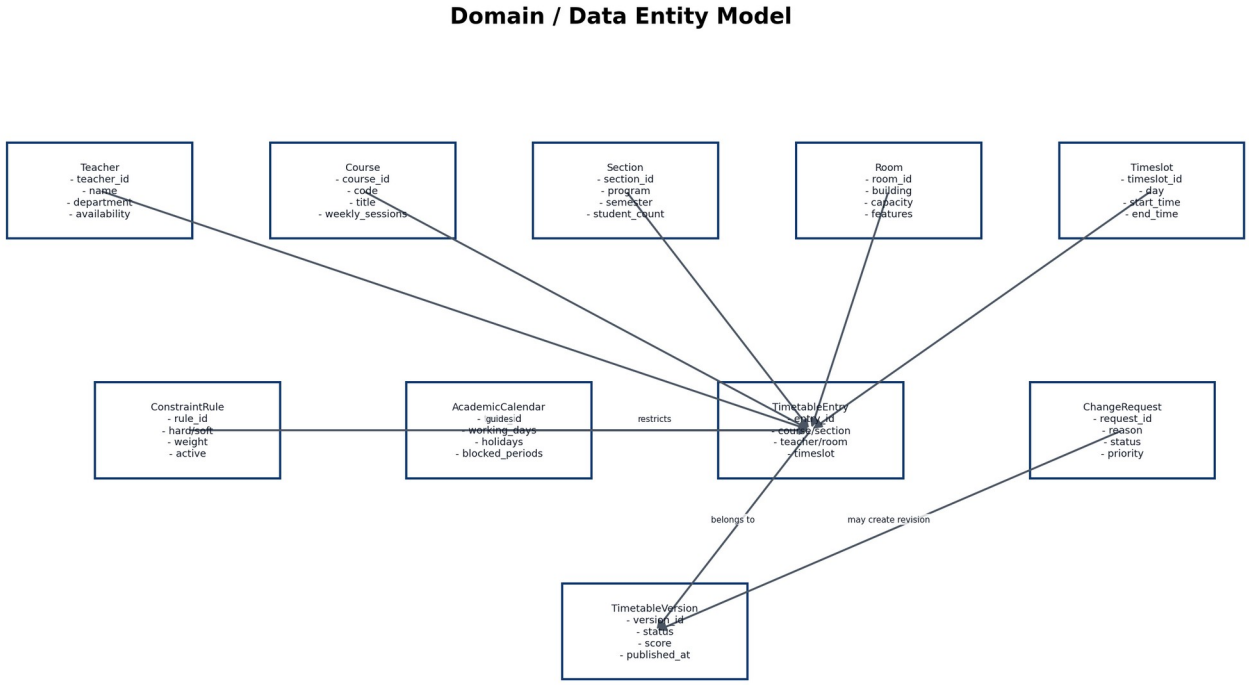


Figure 8. Conceptual domain/data entity model.

Figure 8 identifies the main data concepts that the system must store or process. The most important object is TimetableEntry, which connects course, teacher, section, room, and timeslot under rules and calendar restrictions. TimetableVersion and ChangeRequest support review, publication, and repair after changes.

12. Traceability and Test Planning

12.1 Requirement Traceability Matrix

Use Case	Linked FR IDs	Verification Focus	Related Diagrams
UC-01 Maintain Scheduling Data	FR-01 to FR-05	Create/update records; validate required fields and duplicates.	Context, DFD, Use Case
UC-02 Generate Timetable	FR-06 to FR-11	Generate draft; prevent teacher, room, and section clashes.	Use Case, SSD, Architecture
UC-03 Review and Adjust Timetable	FR-12 to FR-16	Show conflicts, score quality, validate manual edits, lock entries.	Use Case, Swimlane, DFD
UC-04 Publish and View Timetable	FR-17 to FR-21	Publish approved version; allow teacher/student filtered views and export.	Use Case, Architecture
UC-05 Manage Change Requests	FR-22 to FR-26	Submit, approve/reject, re-optimize, version, and compare changes.	Use Case, Swimlane, Domain Model

12.2 Acceptance Test Ideas

Test ID	Scenario	Expected Result
AT-01	Admin adds a course with valid name, code, weekly sessions, and department.	Course is saved and appears in scheduling data.
AT-02	Admin tries to generate timetable while a teacher has two classes at the same timeslot.	System prevents clash in generated timetable or reports infeasibility.
AT-03	Admin manually moves a class to an occupied room and timeslot.	System rejects the change or shows a clear conflict warning.
AT-04	Teacher submits a change request for an assigned class.	Request is stored and visible to Admin/Department Head.
AT-05	Admin publishes an approved timetable.	Teacher and Student views show the published version, not draft versions.
AT-06	Admin re-optimizes after a teacher change request with locked entries.	Locked entries remain unchanged and a new version is created.

12.3 Risks and Mitigation

Risk	Impact	Mitigation
Incorrect input data	Timetable generation fails or produces poor results.	Add validation, required fields, duplicate checks, and data review before generation.
Too many hard constraints	No feasible timetable can be generated.	Show infeasibility reasons and allow the Admin to relax selected rules if they are actually soft preferences.
Unclear actor permissions	Users may access wrong functions.	Define role-based access control early and test each role separately.
Algorithm complexity	Generation may take too long.	Start with a small department and set solver time limits.
Late teacher/room changes	Published timetable becomes outdated.	Use versioning, change requests, locking, and re-optimization.

13. References

- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson.
- Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach. McGraw-Hill.
- Larman, C. (2004). Applying UML and Patterns (3rd ed.). Prentice Hall.
- Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The Unified Modeling Language User Guide (2nd ed.). Addison-Wesley.
- Di Gaspero, L., McCollum, B., & Schaerf, A. (2007). Curriculum-Based Course Timetabling, International Timetabling Competition Track 3 specification.
- UniTime. University Timetabling and Course Management documentation.
- Google OR-Tools. Scheduling and CP-SAT solver documentation.
- Timefold Solver. School timetabling constraints and hard/soft score documentation.

Appendix A. Visio Drawing Guide

A.1 Use Case Diagram Visio Instructions

Create a UML Use Case Diagram. Draw one system boundary rectangle labelled "University Timetable Optimization System". Place Admin, Teacher, Student, and Department Head outside the boundary. Inside the boundary draw five ovals: UC-01 Maintain Scheduling Data, UC-02 Generate Timetable, UC-03 Review & Adjust Timetable, UC-04 Publish / View Timetable, and UC-05 Manage Change Requests. Connect actors only to the use cases they participate in. Do not place actors inside the system boundary. Avoid questionable include/extend relations; use notes for business dependencies if needed.

A.2 System Sequence Diagram Visio Instructions

Create two lifelines: Admin on the left and System on the right. Use horizontal arrows from Admin to System for user actions and arrows from System to Admin for responses. Show the UC-02 flow: select Generate Timetable, display settings, confirm term/data/rules, validate input, show progress/errors, run generation, return draft timetable, save draft, and open review screen. Do not include database or solver as lifelines if the assignment asks for a simple system sequence diagram with only user and system.

A.3 Architecture Diagram Visio Instructions

Draw four component boxes: Frontend, Backend API, Database, and Solver Engine. Connect Frontend to Backend API with "HTTPS/REST". Connect Backend API to Database with "SQL queries" and "records/results". Connect Backend API to Solver Engine with "Problem model" and Solver Engine back to Backend API with "Assignment result". Add a note that users do not directly access the solver engine.